3連LEDライト　フローチャート（main と 10ms タイマ割り込み）

仕様
1. LED が3つ、スイッチが1つ
2. スイッチは 10ms ごとのタイマ割り込みで読む
3. 3回連続で同じ値を読んだら ON／OFF を確定する（チャタリング防止）
4. 3秒未満で離したら短押し。3秒押し続けたら長押し
5. 短押し2回ごとに、点いている LED を消してすぐ次を点灯（LED1 → LED2 → LED3）
6. LED3 の次（2周目）は LED3 を消してすぐ、全 LED が 0.5秒ごとに点滅
   （消灯から開始）。点滅中は短押し無効
7. 点滅中に 3秒長押しすると全消灯に戻り、短押しが有効になる

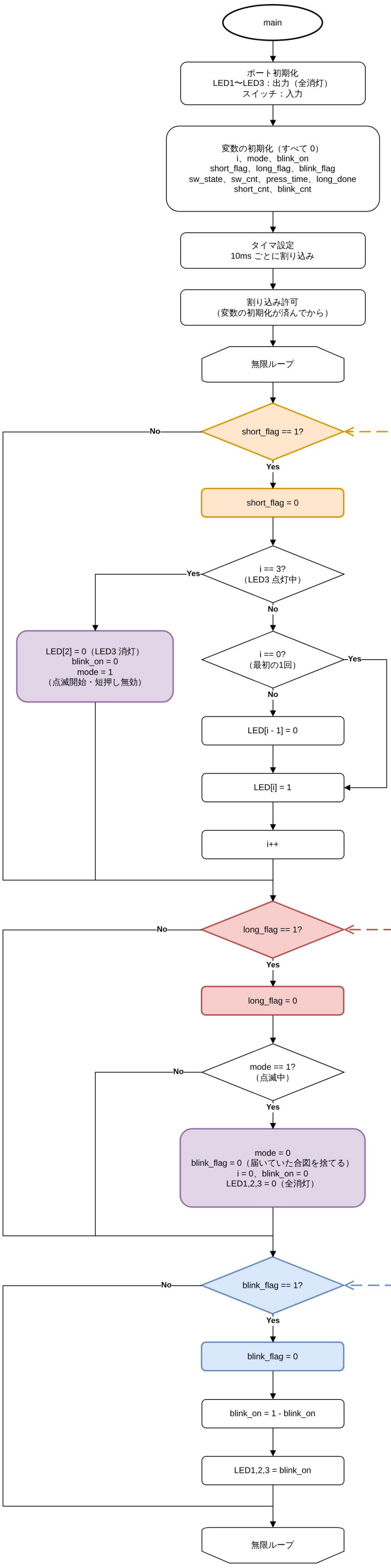
凡例

端子　処理　判断　ループ始端　ループ終端　定義済み処理

short_flag（短押し2回）
long_flag（3秒長押し）
blink_flag（0.5秒経過）
mode（main が書き、割り込みが読む）

- - - ▷　破線の矢印＝割り込みが立てた合図を main が受け取る（制御の流れではない）
同じ色の箱どうしが、同じ変数を書く／読む処理

main（メイン処理）

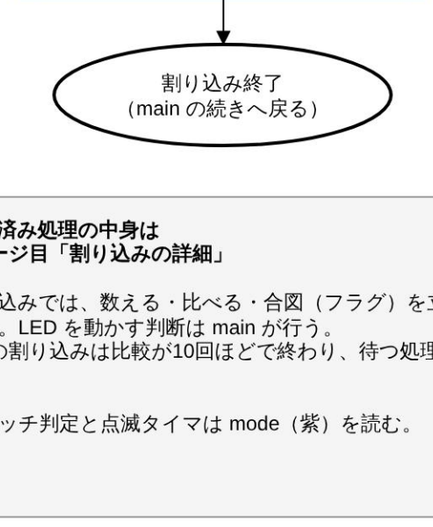


合図（共有変数）

割り込みは main のどの処理の途中でも 10ms ごとに入り、終わると main の続きに戻る

short_flag
long_flag
blink_flag

10ms タイマ割り込み



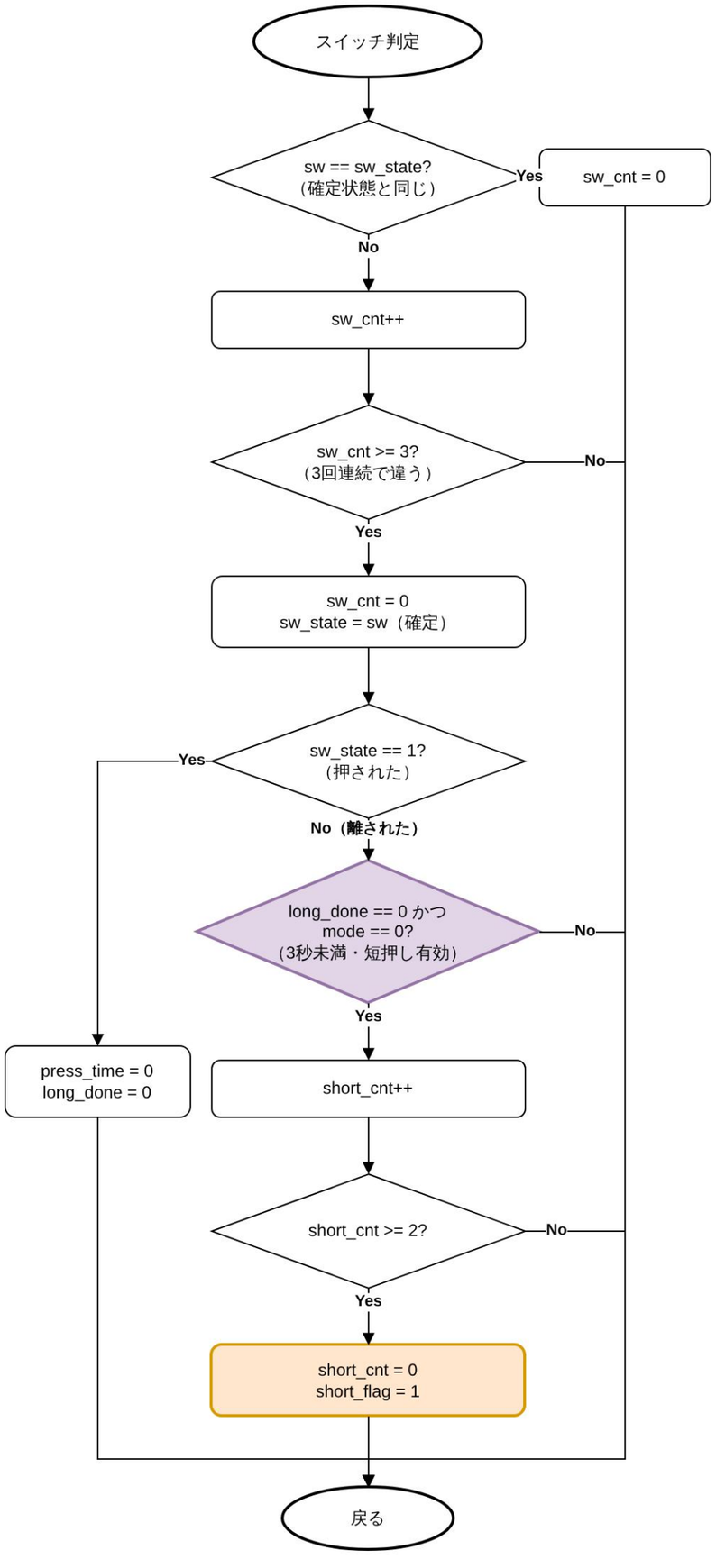
定義済み処理の中身は
2ページ目「割り込みの詳細」

割り込みでは、数える・比べる・合図（フラグ）を立てるだけ。LED を動かす判断は main が行う。
1回の割り込みは比較が10回ほどで終わり、待つ処理はない。

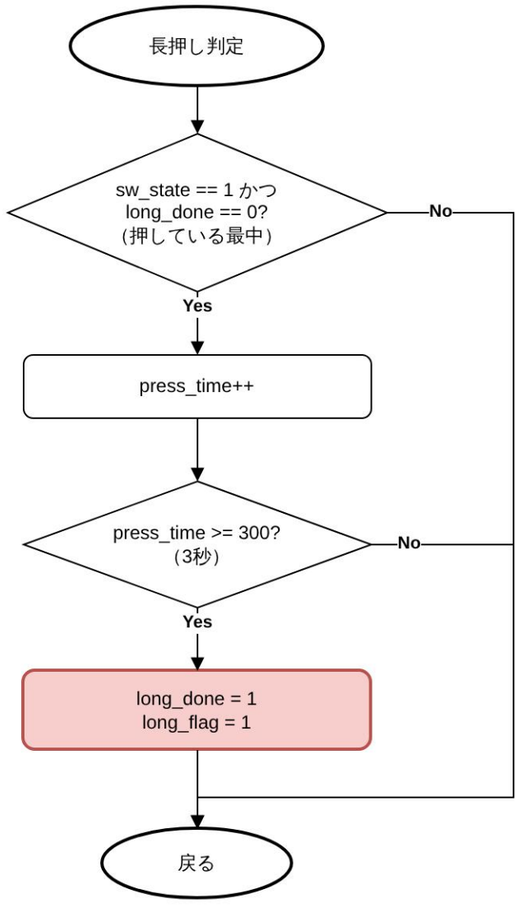
スイッチ判定と点滅タイマは mode（紫）を読む。

割り込みの詳細（1ページ目の定義済み処理の中身）

スイッチ判定（チャタリング防止・短押し）

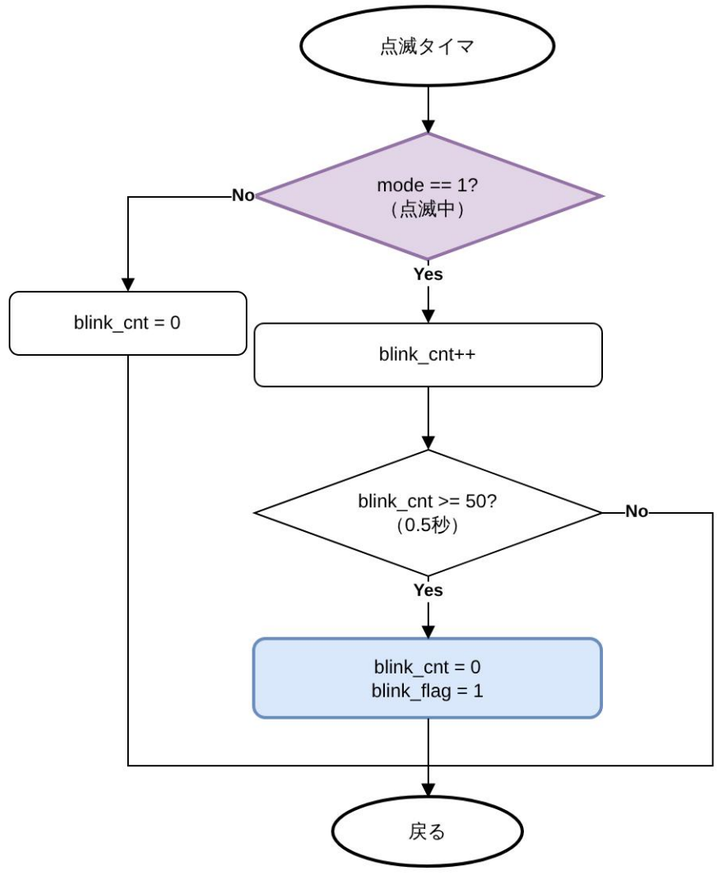


長押し判定



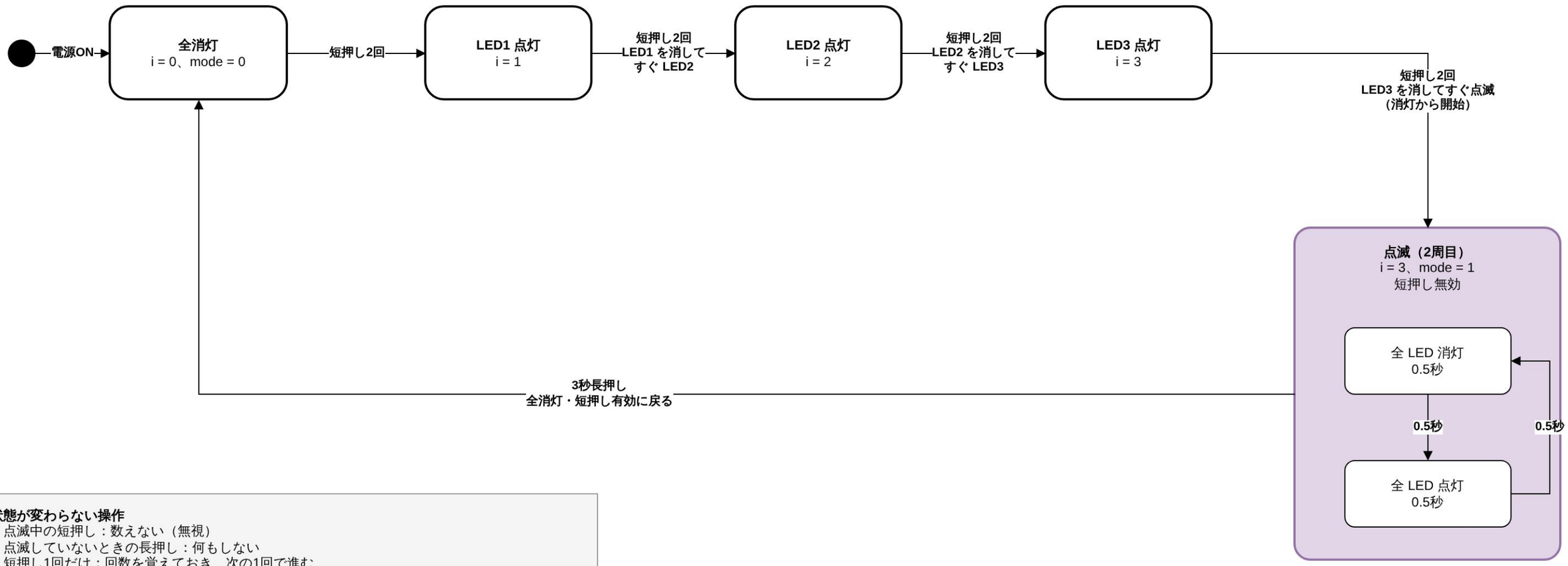
長押しのあと離しても短押しにならないしくみ
3秒に達した時点で long_done = 1。離したときのスイッチ判定で long_done == 0 かつ mode == 0? が No になり、数えない。

点滅タイマ



点滅を消灯から始めるしくみ
通常のあいだは毎回 blink_cnt = 0 に戻しておく。main が mode = 1 にした次の割り込みから数え始めるので、最初の消灯もちょうど 0.5秒になる。

状態遷移図（スイッチ操作と LED の変化）



状態が変わらない操作
・点滅中の短押し：数えない（無視）
・点滅していないときの長押し：何もしない
・短押し1回だけ：回数を覚えておき、次の1回で進む

i と mode の値で状態が決まる
フローチャートの変数がどの状態に当たるかを、各箱の下段に書いています。

動作トレース

このフローチャートを C にそのまま書き写した flow_check.c を動かし、実際に出た記録です。操作：短押し（200ms）を8回 → 点滅中に短押し1回 → 3.5秒押し続けて離す。時刻は最初に押してからの ms。LED は左から LED1・LED2・LED3（● 点灯）。main の行は黄色。

時刻(ms)	10ms タイマ割り込み	main	LED
20	押下が確定（sw_state = 1）		○ ○ ○
220	離したことが確定（sw_state = 0、押下時間 200ms）		○ ○ ○
220	短押し → short_cnt = 1		○ ○ ○
320	押下が確定（sw_state = 1）		○ ○ ○
520	離したことが確定（sw_state = 0、押下時間 200ms）		○ ○ ○
520	短押し → short_cnt = 2		○ ○ ○
520	short_cnt = 0、short_flag = 1		○ ○ ○
520		short_flag を処理 → LED1 点灯（i = 1）	● ○ ○
620	押下が確定（sw_state = 1）		● ○ ○
820	離したことが確定（sw_state = 0、押下時間 200ms）		● ○ ○
820	短押し → short_cnt = 1		● ○ ○
920	押下が確定（sw_state = 1）		● ○ ○
1120	離したことが確定（sw_state = 0、押下時間 200ms）		● ○ ○
1120	短押し → short_cnt = 2		● ○ ○
1120	short_cnt = 0、short_flag = 1		● ○ ○
1120		short_flag を処理 → LED2 点灯（i = 2）	○ ● ○
1220	押下が確定（sw_state = 1）		○ ● ○
1420	離したことが確定（sw_state = 0、押下時間 200ms）		○ ● ○
1420	短押し → short_cnt = 1		○ ● ○
1520	押下が確定（sw_state = 1）		○ ● ○
1720	離したことが確定（sw_state = 0、押下時間 200ms）		○ ● ○
1720	短押し → short_cnt = 2		○ ● ○
1720	short_cnt = 0、short_flag = 1		○ ● ○
1720		short_flag を処理 → LED3 点灯（i = 3）	○ ○ ●
1820	押下が確定（sw_state = 1）		○ ○ ●
2020	離したことが確定（sw_state = 0、押下時間 200ms）		○ ○ ●
2020	短押し → short_cnt = 1		○ ○ ●
2120	押下が確定（sw_state = 1）		○ ○ ●
2320	離したことが確定（sw_state = 0、押下時間 200ms）		○ ○ ●
2320	短押し → short_cnt = 2		○ ○ ●
2320	short_cnt = 0、short_flag = 1		○ ○ ●
2320		short_flag を処理 → LED3 消灯、mode = 1（点滅開始・短押し無効）	○ ○ ○
2420	押下が確定（sw_state = 1）		○ ○ ○
2620	離したことが確定（sw_state = 0、押下時間 200ms）		○ ○ ○
2620	短押しだが mode = 1（点滅中）なので数えない		○ ○ ○
2820	blink_cnt = 50（0.5秒） → blink_flag = 1		○ ○ ○
2820		blink_flag を処理 → 全LED 点灯	● ● ●
3320	押下が確定（sw_state = 1）		● ● ●
3320	blink_cnt = 50（0.5秒） → blink_flag = 1		● ● ●
3320		blink_flag を処理 → 全LED 消灯	○ ○ ○
3820	blink_cnt = 50（0.5秒） → blink_flag = 1		○ ○ ○
3820		blink_flag を処理 → 全LED 点灯	● ● ●
4320	blink_cnt = 50（0.5秒） → blink_flag = 1		● ● ●
4320		blink_flag を処理 → 全LED 消灯	○ ○ ○
4820	blink_cnt = 50（0.5秒） → blink_flag = 1		○ ○ ○
4820		blink_flag を処理 → 全LED 点灯	● ● ●
5320	blink_cnt = 50（0.5秒） → blink_flag = 1		● ● ●
5320		blink_flag を処理 → 全LED 消灯	○ ○ ○
5820	blink_cnt = 50（0.5秒） → blink_flag = 1		○ ○ ○
5820		blink_flag を処理 → 全LED 点灯	● ● ●
6310	press_time = 300（3秒） → long_done = 1、long_flag = 1		● ● ●
6310		long_flag を処理 → mode = 0、全消灯（短押し有効に戻る）	○ ○ ○
6820	離したことが確定（sw_state = 0、押下時間 3000ms）		○ ○ ○

変数一覧

変数	値を書く側	意味	初期値	元の図での名前
sw_state	割り込み	スイッチの確定状態（1 = 押されている）	0	—
sw_cnt	割り込み	確定状態と違う値が続いた回数（3 で確定）	0	cnt（前半の役割）
press_time	割り込み	押している時間（10ms 単位、300 = 3秒）	0	cnt（後半の役割）
long_done	割り込み	今回の押下で長押しを判定済みか	0	—
short_cnt	割り込み	短押しの回数（2 回で short_flag を立てる）	0	tanoshi
blink_cnt	割り込み	点滅用の時間（10ms 単位、50 = 0.5秒）	0	—
short_flag	割り込みが 1、main が 0	短押し2回の合図	0	Tflag
long_flag	割り込みが 1、main が 0	3秒長押しの合図	0	nagaoshi
blink_flag	割り込みが 1、main が 0	0.5秒経過の合図	0	—
mode	main（割り込みが読む）	0 = 通常、1 = 点滅（短押し無効）	0	（短押し割り込みの無効化）
i	main	次に点ける LED の番号（0〜2）。3 = LED3 点灯中	0	i（同じ意味）
blink_on	main	点滅の今の状態（1 = 点灯）	0	—
LED[0]〜LED[2]	main	LED1〜LED3 の出力（1 = 点灯）	0	LED[ ]

・変数はすべてグローバル変数（割り込みをまたいで値を残すため。関数の中のローカル変数だと毎回 0 に戻る）
・割り込みと main の両方から使う変数（short_flag、long_flag、blink_flag、mode）は、C では volatile を付ける
・合図を受け取ったら main がすぐ 0 に戻す。戻さないと同じ合図を何度も処理してしまう

仕様との対応

仕様	図のどこで実現しているか
1. LED 3つ、スイッチ 1つ	main：ポート初期化
2. 10ms ごとのタイマ割り込みでスイッチを読む	main：タイマ設定・割り込み許可 ／ 割り込み：sw = スイッチ入力ポートの値 → スイッチ判定
3. 3回連続で同じ値なら ON／OFF を確定	スイッチ判定：sw == sw_state? → sw_cnt++ → sw_cnt >= 3?（違う値が続いた回数を、割り込みをまたいで数える）
4. 3秒未満で離したら短押し	スイッチ判定：離したことが確定した時点で long_done == 0 かつ mode == 0? を見る（押した瞬間には数えない）
4. 3秒押し続けたら長押し	長押し判定：sw_state == 1 かつ long_done == 0? → press_time++ → press_time >= 300?（離すのを待たない）
5. 短押し2回で次の LED（前を消してすぐ次）	スイッチ判定：short_cnt >= 2? → short_flag = 1 ／ main：i == 0? → LED[i-1] = 0 → LED[i] = 1 → i++
6. 2周目は全 LED が 0.5秒ごとに点滅（消灯から）	main：i == 3? → LED[2] = 0、blink_on = 0、mode = 1 ／ 点滅タイマ：blink_cnt >= 50? → blink_flag = 1 ／ main：blink_on を反転して出力
6. 点滅中は短押し無効	スイッチ判定：long_done == 0 かつ mode == 0? が No なら数えない（割り込み自体は止めない。止めると長押しも判定できなくなる）
7. 点滅中に 3秒長押しで短押し有効に戻る	main：long_flag == 1? → mode == 1? → mode = 0、blink_flag = 0、i = 0、全消灯

確認方法
flow_check.c（このフローチャートを C に書き写したもの）と、テスト済みの light.c に同じスイッチ入力を与え、10ms ごとに LED の出力を比べた。ランダムな短押し・長押し・チャタリングで 33,094,692 周期（約 92 時間ぶん）すべて一致。
わざと図を間違えた版（フラグを 0 に戻さない、長押しの閾値を 3 にする、点滅中も短押しを数える など）は、すべて不一致として検出できた。